

Interpaws Updated Master Plan (v2.0)

Last Updated: November 11, 2025

Overview

The initial 18-day sprint goal of achieving a feature-complete MVP has been met ahead of schedule [cite:

computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/README.md,

computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/DAYS_12-14_SUMMARY.md]. The full-stack application, including dual-role authentication and complete admin CRUD for staff, surgeries, and medications, is implemented.

This updated plan details the next two phases of development:

Phase 1: The "Launch" Plan: Focuses on implementing the AI "learning loop," hardening the application, and deploying it for the Thanksgiving demo.

Phase 2: The "VPMS 2.0" Sprints: Outlines the advanced, next-generation AI features to be developed post-launch that will set Interpaws apart from all existing VPMS solutions.

Phase 1: The "Launch" Plan (Now → Nov 27)

Goal: Deploy a polished, "learning" application for the Thanksgiving demo.

Sprint 1 (Days 10-11): Implement the "AI Learning Loop"

Goal: Make the AI's recommendations genuinely improve over time by learning from successful, completed bookings.

Task 1 (Backend): Enhance POST /suggest_slots

Modify the endpoint in backend/app/main.py [cite:

computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/backend/app/main.py].

When a new complaint vector is received, first perform a vector search against the AIFeedbackLog table [cite:

computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/backend/app/models.py] to find the staff_ids of vets who have successfully treated similar cases.

Modify the main staff skills query to boost the score (e.g., L2 distance) of these "proven" staff members, blending past success with current skill matching.

Task 2 (Backend): Enhance POST /chat

Modify the endpoint in backend/app/main.py [cite:

computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/backend/app/main.py].

Inject this "proven staff" information into the LLM prompt.

Example Prompt: "Context: The user's complaint is '...'. Our system shows that Dr. Smith and Dr. Jones have successfully treated similar cases in the past. The user is asking: '...'"

Task 3 (Frontend): Automate the Feedback Loop

Modify `handleUpdateStatus` in `frontend/src/components/AdminBookingList.js` [cite: `computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/frontend/src/components/AdminBookingList.js`].

Add logic: if (`newStatus === "completed"`), automatically make a POST request to the `/api/log-feedback/` endpoint [cite:

`computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/backend/app/main.py`] using the `booking.id`.

This one change closes the entire loop: An admin confirms a successful appointment, and the AI instantly learns from it.

Sprint 2 (Days 12-14): Hardening & E2E Testing

Goal: Polish the UI and execute the full E2E test plan to ensure production readiness.

Task 1 (Bug Hunt): Formally execute every step in `E2E_TESTING_GUIDE.md` [cite: `computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/E2E_TESTING_GUIDE.md`] for both Client and Admin workflows.

Task 2 (New Test Case): Add a test to the E2E guide to validate the "Learning Loop":

Book an appointment for "limping dog" with Dr. A.

As admin, mark the booking "completed."

As a new client, submit a complaint for "sore paw."

Verify: Dr. A now appears as a top recommendation, validating the loop.

Task 3 (Polish): Fix all bugs. Implement UI polish (loading spinners, error messages) on all client and admin pages, as planned in the original master plan [cite: `Interpaws masterplan .pdf`].

Sprint 3 (Days 15-17): Deployment & Live Data

Goal: Get the application live on the internet and populate it with rich data.

Task 1 (Deploy):

Provision a cloud PostgreSQL database (e.g., Render, Heroku, Aiven).

Deploy Backend (FastAPI) to Render/Heroku and Frontend (Next.js) to Vercel [cite: `Interpaws masterplan .pdf`].

Configure all production environment variables (`DATABASE_URL`, `SECRET_KEY`, `OLLAMA_HOST`) in the deployment services.

Task 2 (Live Testing): Test the live, deployed application. Fix any deployment-specific bugs (CORS, DB permissions, env var issues) [cite: `Interpaws masterplan .pdf`].

Task 3 (Run Simulator):

Run the `seed.py` script [cite:

`computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/backend/app/seed.py`] one time against the live database.

This populates the DB with hundreds of realistic clients, pets, and bookings, providing immediate, rich data for the AI to "learn" from [cite: Interpaws masterplan .pdf].

Sprint 4 (Days 18-20): Demo Prep & Final Buffer

Goal: Prepare a flawless demo and have buffer time before Thanksgiving.

Task 1 (Demo Script): Write a step-by-step presentation script [cite: Interpaws masterplan .pdf].

Task 2 (Practice): Practice the demo script on the live, populated application.

Task 3 (Buffer): Final emergency bug fixes.

Phase 2: The "VPMS 2.0" Sprints (Post-Thanksgiving)

Goal: Implement next-generation AI features that are not present in any standard VPMS.

Sprint 5 (Post-Launch): AI-Triage & Complaint Augmentation

Goal: Use the chatbot to intelligently "interview" the client before they ask for recommendations, improving data quality.

Backend: Create a new endpoint, `/api/chat/triage`, that takes a basic complaint and asks clarifying questions.

Frontend: Modify `ClientBookingForm.js` [cite:

`computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/frontend/src/components/ClientBookingForm.js`]. When a user enters a complaint, it automatically triggers `AIChat.js` [cite:

`computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/frontend/src/components/AIChat.js`] to start the triage interview.

Integration: The "Get AI Suggestions" button will now bundle the entire chat history into the `complaint_text` sent to `/api/suggest_slots`, resulting in a much richer vector and better matches.

Sprint 6 (Post-Launch): AI-Powered Surgical Inventory Prediction

Goal: Connect surgery scheduling to the medication inventory to prevent conflicts.

Backend (New Model): Create a new `SurgeryInventoryLink` model (`models.py`) that maps a `surgery_type` (e.g., "Spay") to a list of Medication [cite:

`computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/backend/app/models.py`] IDs and required quantities.

Backend (New Endpoint): Create GET `/api/surgeries/check_inventory?type=....` This endpoint checks the `stock_quantity` in the Medications table [cite:

`computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/backend/app/models.py`] for all items linked to the `surgery_type`.

Frontend: Modify `admin/surgeries/page.js` [cite:

`computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/frontend/src/app/admin/surgeries/page.js`]. When an admin selects a "Surgery Type," it calls the new endpoint and displays a checklist of required inventory, highlighting low-stock items.

Integration (Bonus): Modify PUT `/api/surgeries/{id}` [cite:

`computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf`

4ebf48813e54d26edac/backend/app/main.py]. When a surgery status is set to "Completed," automatically decrement the linked medications from inventory.

Sprint 7 (Post-Launch): Proactive Client Wellness Outreach

Goal: Have the AI proactively find and message clients who are due for checkups.

Backend: Create a new script (backend/app/wellness_outreach.py) to run as a nightly job.

Backend Logic: The script scans pets and bookings [cite:

computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/backend/app/models.py] for at-risk pets. For each match, it finds open slots [cite:

computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/backend/app/booking_logic.py] and queries the client's preferences vector [cite:

computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/backend/app/models.py] (e.g., for "prefers mornings").

Integration: The script feeds this data (Pet, Client, Preferred Slot) into

get_ollama_recommendation [cite:

computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/backend/app/ai_services.py] to generate a personalized email. (Note: This sprint will require adding an email-sending service like SendGrid or AWS SES).

Sprint 8 (Post-Launch): Dynamic Slot-Filling (Admin AI Assistant)

Goal: Help admins fill cancelled slots intelligently and optimize the schedule.

Backend: Create a new endpoint, GET /api/admin/cancellation_suggestion/{booking_id}.

Backend Logic: When a booking is cancelled, this endpoint scans bookings for appointments later in the day and uses vector search on the preferences table [cite:

computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/backend/app/models.py] to find clients who "prefer earlier appointments."

Frontend: Modify AdminBookingList.js [cite:

computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/frontend/src/components/AdminBookingList.js]. When handleUpdateStatus is set to "cancelled," it calls the new endpoint.

Integration: A new "Suggestion" card appears on the admin/dashboard/page.js [cite:

computerengineeringkid/interpaws/computerengineeringkid-Interpaws-466bfae3925387dfaa2bf4ebf48813e54d26edac/frontend/src/app/admin/dashboard/page.js]: "A 2 PM slot just opened. [Client Name] is booked at 4 PM and prefers earlier times. [Offer Slot] / [Dismiss]".